

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное автономное образовательное учреждение высшего образования
«Дальневосточный федеральный университет»
(ДВФУ)

ИНСТИТУТ МАТЕМАТИКИ И КОМПЬЮТЕРНЫХ ТЕХНОЛОГИЙ

Департамент математики и компьютерного моделирования

Отчет
к лабораторной работе №1 по дисциплине «Суперкомпьютеры и параллельное программирование»

По теме: «Применение делегатов»

Вариант 20

Выполнил: студент группы Б9122-01.03.02мкт

_____ Чирич И.С.
подпись *Ф.И.О.*

Проверил: Доцент

_____ Селезнев Т.Э.
подпись *Ф.И.О*

г. Владивосток
2025

Содержание

Постановка задачи **3**

Индивидуальное задание **4**
Написание программного кода 4
Тестирование 6

Вопросы для защиты работы **8**

Выводы **9**

Постановка задачи

Целью данной лабораторной работы является изучение возможности запуска потоков с использованием делегатов.

Задачи лабораторной работы:

- Научиться объявлять, инициализировать и запускать потоки с использованием пользовательских делегатов
- Научиться запускать потоки с использованием библиотечных делегатов `Action<T>` и `Func<T>`
- Научиться запускать параллельные потоки с использованием лямбда-выражений

Индивидуальное задание

Написание программного кода

Создадим шаблон консольного проекта при помощи утилиты `dotnet`:

```
dotnet new console -n async_delegates
```

В корне проекта создастся файл Project.cs, в котором и будет написан код лабораторной работы.

Создаем класс Program с методом Main:

```
class Program
{
        static void Main(string[] args)
        {

        }
}
```

В методе Main опишем следующую логику:

- Определить переменную типа делегата `Func<int, List<int> >`
- Определить переменную типа `IAsyncResult` для получения результата вычисления делегата
- Запустить делегат на ассинхронное выполнение с помощью метода `BeginInvoke`
- Опишем цикл, который раз в 100 мс опрашивает объект интерфейса о завершении работы, делая анимацию в консоли
- В конце получаем результирующий массив, выводим результирующие данные.

```
static void Main(string[] args)
{
        string[] SYMBOLS = { "-", "\\", "|", "/"};
        int SIZE = 1_000_000;
        Func<int, List<int> > dl = DoHardWork;
        IAsyncResult ar = dl.BeginInvoke(SIZE, null, null);
        Console.WriteLine("Вычисления запущены...");
        for (int i = 0; !ar.IsCompleted ; i++)
        {
                Console.Write('\b');
                Console.Write(SYMBOLS[i % 4]);
                Thread.Sleep(100);
        }
        List<int> result = dl.EndInvoke(ar);
        Console.WriteLine("\Вычисленияп выполнены.");
        Console.WriteLine("Длина полученного массива: {0}", result.Count());
```

```
17          Console.WriteLine("Длина входного массива: {0}", SIZE);
18          Console.ReadKey();
19  }
```

Важно уточнить о массиве строк SYMBOLS. Этот массив состоит из строк, которые меняются в выводе консоли во время выполнения ассинхронной задачи. Такая консольная анимация необходима в рамках поставленной задачи - демонстрация выполнения сложных вычислений в отдельном потоке без блокировки пользовательского интерфейса.

Далее опишем статический метод DoHardWork, который и содержит симуляцию сложных вычислений. В соответствии с заданием, необходимо по полученному размеру массива сгенерировать его, затем - отфильтровать по условию делимости на 6:

```
1   static List<int> DoHardWork(int size)
2   {
3           int MAX_VALUE = 20_000_000;
4           Random random = new Random();
5           List<int> list = new List<int>();
6           for (int i = 0; i < size; i++)
7           {
8                   list.Add(random.Next(MAX_VALUE));
9           }
10          List<int> newList = new List<int>();
11          for (int i = 0; i < size; i++)
12          {
13                  if (list[i] % 6 == 0)
14                  {
15                          newList.Add(list[i]);
16                  }
17          }
18          // делаем' остановку' на 3000ms, чтобы' симулировать' сложную' работу'
19          Thread.Sleep(3000);
20          return newList;
21  }
```

Итоговый код в листинге ниже:

```
1   using System;
2   using System.Collections.Generic;
3   using System.Linq;
4   using System.Text;
5   using System.Threading;
6   using System.Threading.Tasks;
7
8   namespace async_delegates
9   {
10          internal class Program
11          {
12                  static void Main(string[] args)
```

```
13              {
14                      string[] SYMBOLS = { "-", "\\", "|", "/"};
15                      int SIZE = 1_000_000;
16                      Func<int, List<int> > dl = DoHardWork;
17                      IAsyncResult ar = dl.BeginInvoke(SIZE, null, null);
18                      Console.WriteLine("Вычисления запущены...");
19                      for (int i = 0; !ar.IsCompleted ; i++)
20                      {
21                              Console.Write('\b');
22                              Console.Write(SYMBOLS[i % 4]);
23                              Thread.Sleep(100);
24                      }
25                      List<int> result = dl.EndInvoke(ar);
26                      Console.WriteLine("\Вычислениян выполнены.");
27                      Console.WriteLine("Длина полученного массива: {0}", result.
                            Count());
28                      Console.WriteLine("Длина входного массива: {0}", SIZE);
29                      Console.ReadKey();
30              }
31              //
32              static List<int> DoHardWork(int size)
33              {
34                      int MAX_VALUE = 20_000_000;
35                      Random random = new Random();
36                      List<int> list = new List<int>();
37                      for (int i = 0; i < size; i++)
38                      {
39                              list.Add(random.Next(MAX_VALUE));
40                      }
41                      List<int> newList = new List<int>();
42                      for (int i = 0; i < size; i++)
43                      {
44                              if (list[i] % 6 == 0)
45                              {
46                                      newList.Add(list[i]);
47                              }
48                      }
49                      // делаем' остановку' на' 3000ms, чтобы' симулировать' сложную'
                            работу
50                      Thread.Sleep(3000);
51                      return newList;
52              }
53      }
54  }
```

Тестирование

Запуск программы осуществим командой, которая скомпилирует проект и запустит его в среде выполнения:

```
1  dotnet build; dotnet run
```

Вывод программы:

```
Вычисления запущены...
-
Вычисления выполнены.
Длина полученного массива: 166685
Длина входного массива: 1000000
```

Данный вывод говорит о том, что программный код работает. Вычисления осуществляются в отдельном потоке, ассинхронно с пользовательским вводом.

Вопросы для защиты работы

1. **Поясните назначение типа `IAsyncResult`.**

   `IAsyncResult` - тип, который позволяет получить информацию о выполняющемся в отдельном потоке делегате. Мы не можем просто получить результат через метод `EndInvoke`, потому что результат вычисления может не успеть за его запросом, поэтому вызывающий поток будет вынужден простаивать в ожидании, что не эффективно.

2. **Для чего используется метод `Thread.Sleep()`?** Метод `Sleep` класса `Thread` позволяет текущему потоку приостановить выполнение на заданное количество миллисекунд (если точнее, то на как минимум указанное количество миллисекунд).

3. **Поясните механизм возврата значения из метода асинхронного делегата**

   Механизм описывается в три шага:

   - Вызов метода `BeginInvoke`
   - (Опционально, асинхронный метод) использование интерфейса `IAsyncResult` для получения статуса выполнения делегата
   - Вызов метода `EndInvoke` для получения результата.

4. **Как произвести возврат более одного значения из метода?**

   - Возврат структуры с полями, в которые запишутся возвращаемые значения
   - Указание `out` параметров (компилятор заставит объявить их в теле функции)
   - Указание `ref` параметров, что является аналогом передачи значения по ссылке.

5. **Какая разница существует между библиотечными делегатами, пользовательскими типами делегатов и лямбда-выражениями? Являются ли эти делегаты взаимозаменяемыми при реализации асинхронного вызова методов?**

   В контексте работы с асинхронными делегатами разницы нет, они взаимозаменяемы, так как все делегаты унаследованы от `System.delegate`. Если вообще, то библиотечные делегаты - шаблонизированные делегаты. Пользовательские делегаты можно сделать шаблонизированными, но в этом нет необходимости. Лямбда функции - анонимные функции, которые автоматически приводятся к нужному типу делегата.

Выводы

В данной лабораторной работе изучены способы работы с ассинхронными делегатами. Был продемонстрирован листинг, задающий делегат, выполняемый в отдельном потоке без блокировки основного потока.